

Week 11

Task 1: To Connect Modules, Validate Interfaces, Test End-to-End Execution, Handle Errors, Improve Reliability, Refactor Components.

CODE:

```
saichaitanya@GeethzzSai: ~/s × + ▾
GNU nano 8.7.1 1.c *
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* =====
Parser Module
===== */

int parse_command(char *input)
{
    if (input == NULL || strlen(input) == 0)
    {
        printf("Parser: Empty command.\n");
        return 0;
    }

    printf("Parser: Command accepted -> %s\n", input);

    return 1;
}

/* =====
Executor Module
===== */

int execute_command(char *command)
{
    int result;

    printf("Executor: Running command...\n");

    result = system(command);

    if (result != 0)
    {
        printf("Executor: Command failed.\n");
    }
}
```

```
    printf("Executor: Running command...\n");

    result = system(command);

    if (result != 0)
    {
        printf("Executor: Command failed.\n");
        return 0;
    }

    printf("Executor: Command completed successfully.\n");

    return 1;
}

/* =====
Main Program
===== */

int main()
{
    char command[100];

    printf("Enter command: ");

    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    /* Validate command */
    if (!parse_command(command))
    {
        printf("Main: Invalid command.\n");
        return 1;
    }
}
```

```

/* Validate command */
if (!parse_command(command))
{
    printf("Main: Invalid command.\n");
    return 1;
}

/* Execute command */
if (!execute_command(command))
{
    printf("Main: Execution failed.\n");
    return 1;
}

printf("Main: End-to-end execution successful.\n");

return 0;
}
|

```

OUTPUT:

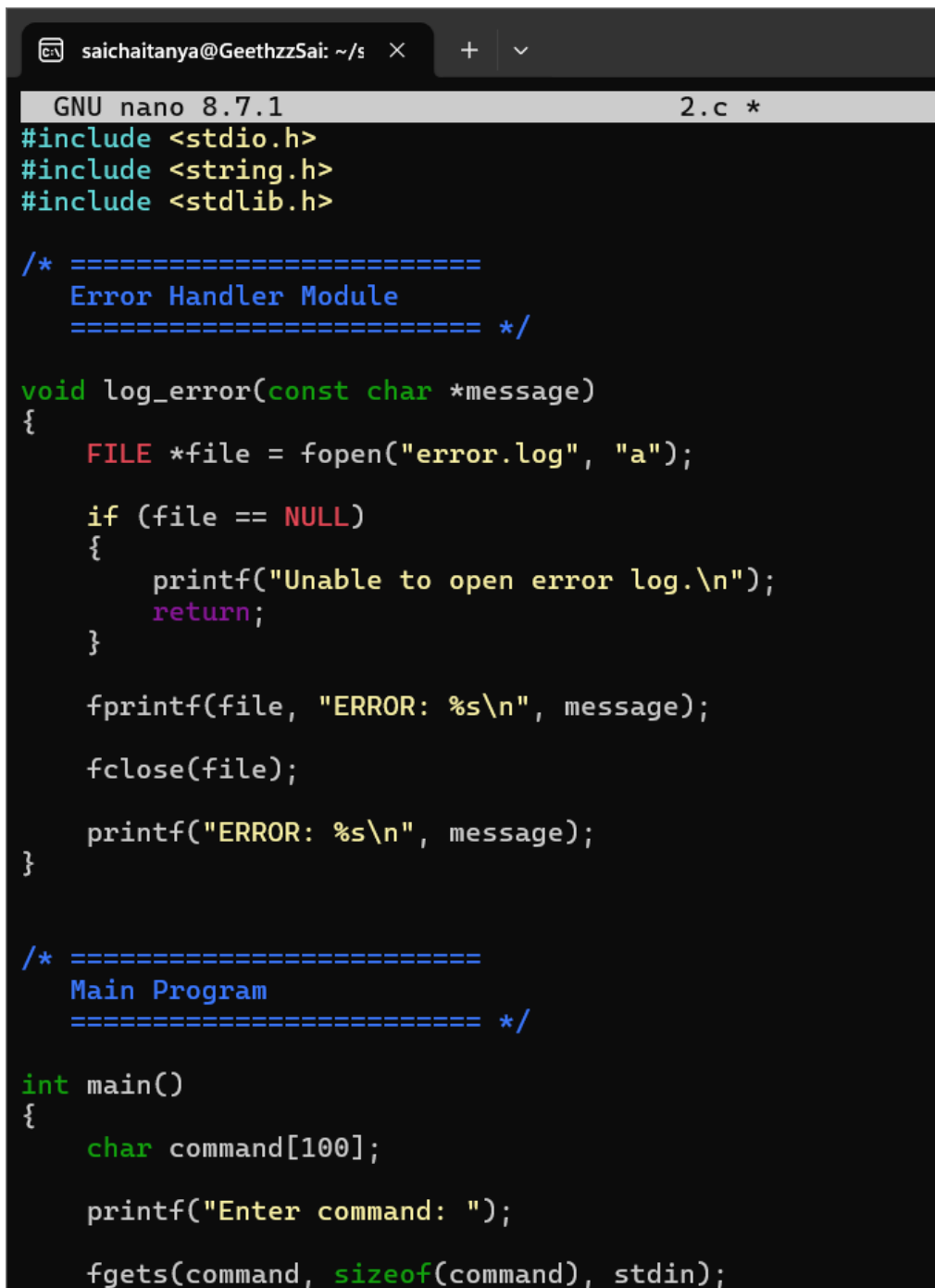
```

saichaitanya@GeethzzSai: ~/s  ×  +  ∨
saichaitanya@GeethzzSai:~$ mkdir skill_week11
saichaitanya@GeethzzSai:~$ cd skill_week11
saichaitanya@GeethzzSai:~/skill_week11$ nano 1.c
saichaitanya@GeethzzSai:~/skill_week11$ gcc 1.c -o 1
saichaitanya@GeethzzSai:~/skill_week11$ ./1
Enter command: ls
Parser: Command accepted -> ls
Executor: Running command...
1 1.c
Executor: Command completed successfully.
Main: End-to-end execution successful.
saichaitanya@GeethzzSai:~/skill_week11$ |

```

Task 2: To Detect Runtime Errors, Generate Messages, Log Failures, Handle Invalid Syntax, Recover Gracefully, Test Error Scenarios.

CODE:



```
GNU nano 8.7.1 2.c *
#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/* =====
Error Handler Module
===== */

void log_error(const char *message)
{
    FILE *file = fopen("error.log", "a");

    if (file == NULL)
    {
        printf("Unable to open error log.\n");
        return;
    }

    fprintf(file, "ERROR: %s\n", message);
    fclose(file);

    printf("ERROR: %s\n", message);
}

/* =====
Main Program
===== */

int main()
{
    char command[100];

    printf("Enter command: ");

    fgets(command, sizeof(command), stdin);
```

```
int main()
{
    char command[100];

    printf("Enter command: ");

    fgets(command, sizeof(command), stdin);

    command[strcspn(command, "\n")] = '\0';

    /* Check for empty command */
    if (strlen(command) == 0)
    {
        log_error("Empty command entered.");

        printf("Recovering: Please enter a command next time.\n");

        return 1;
    }

    /* Check for invalid syntax */
    if (strstr(command, "&&") != NULL)
    {
        log_error("Invalid command syntax.");

        printf("Recovering: Invalid syntax ignored.\n");

        return 1;
    }
}
```

```

/* Check for invalid syntax */
if (strstr(command, "&&") != NULL)
{
    log_error("Invalid command syntax.");

    printf("Recovering: Invalid syntax ignored.\n");

    return 1;
}

/* Execute command */
printf("Executing command...\n");

int result = system(command);

/* Check execution result */
if (result != 0)
{
    log_error("Command execution failed.");

    printf("Recovering: Shell remains active.\n");

    return 1;
}

printf("Command executed successfully.\n");

return 0;
}

```

OUTPUT:

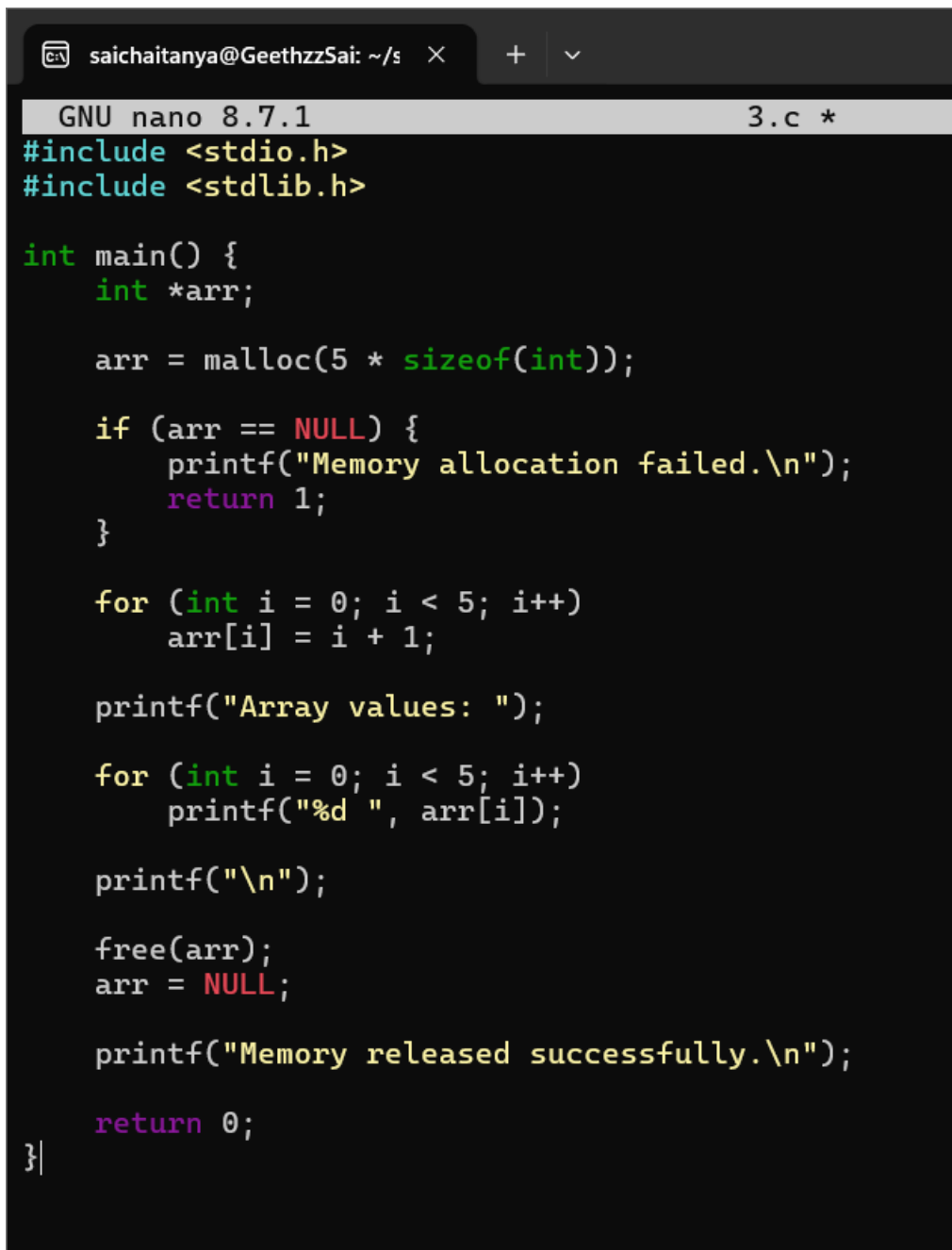
```

saichaitanya@GeethzzSai:~/skill_week11$ nano 2.c
saichaitanya@GeethzzSai:~/skill_week11$ gcc 2.c -o 2
saichaitanya@GeethzzSai:~/skill_week11$ ./2
Enter command: pwd
Executing command...
/home/saichaitanya/skill_week11
Command executed successfully.
saichaitanya@GeethzzSai:~/skill_week11$ ./2
Enter command: ls
Executing command...
1 1.c 2 2.c
Command executed successfully.
saichaitanya@GeethzzSai:~/skill_week11$ |

```

Task 3: To Run Valgrind, Identify Leaks, Fix Memory Errors, Release Resources, Verify Cleanup, Document Findings.

CODE:

A screenshot of a terminal window with a dark background. The terminal title bar shows 'saichaitanya@GeethzzSai: ~/s' and a tab icon. The nano editor header shows 'GNU nano 8.7.1' and '3.c *'. The code is a C program that demonstrates memory allocation and deallocation. It includes `<stdio.h>` and `<stdlib.h>`. The `main` function declares an integer pointer `arr`, allocates memory for 5 integers using `malloc`, checks for allocation failure, initializes the array with values 1 through 5, prints the values, frees the memory using `free`, sets `arr` to `NULL`, prints a success message, and returns 0. The code is color-coded: keywords in green, comments in yellow, and literals in red.

```
GNU nano 8.7.1 3.c *
#include <stdio.h>
#include <stdlib.h>

int main() {
    int *arr;

    arr = malloc(5 * sizeof(int));

    if (arr == NULL) {
        printf("Memory allocation failed.\n");
        return 1;
    }

    for (int i = 0; i < 5; i++)
        arr[i] = i + 1;

    printf("Array values: ");

    for (int i = 0; i < 5; i++)
        printf("%d ", arr[i]);

    printf("\n");

    free(arr);
    arr = NULL;

    printf("Memory released successfully.\n");

    return 0;
}
```

OUTPUT:

```
saichaitanya@GeethzzSai:~/skill_week11$ nano 3.c
saichaitanya@GeethzzSai:~/skill_week11$ gcc 3.c -o 3
saichaitanya@GeethzzSai:~/skill_week11$ ./3
Array values: 1 2 3 4 5
Memory released successfully.
saichaitanya@GeethzzSai:~/skill_week11$ |
```

Task 4: To Design Test Cases, Create Test Scripts, Automate Execution, Verify Outputs, Compare Results, Generate Reports.

CODE:

```
GNU nano 8.7.1 4.c *
#!/bin/bash

echo "==== Memory Test Report ====="

echo "Test 1: Program execution"

output=$(./bin/memory_test)

echo "$output"

if echo "$output" | grep -q "Memory released successfully."; then
    echo "Test 1: PASS"
else
    echo "Test 1: FAIL"
fi

echo

echo "Test 2: Array values"

if echo "$output" | grep -q "1 2 3 4 5"; then
    echo "Test 2: PASS"
else
    echo "Test 2: FAIL"
fi

echo
echo "==== Test Summary ====="
echo "Testing completed."
|
```

OUTPUT:

```
saichaitanya@GeethzzSai:~/skill_week11$  
saichaitanya@GeethzzSai:~/skill_week11$ nano 4.c  
saichaitanya@GeethzzSai:~/skill_week11$ bash 4.c  
===== Memory Test Report =====  
Test 1: Program execution  
4.c: line 7: ./bin/memory_test: No such file or directory  
  
Test 1: FAIL  
  
Test 2: Array values  
Test 2: FAIL  
  
===== Test Summary =====  
Testing completed.  
saichaitanya@GeethzzSai:~/skill_week11$ |
```